SLIDE 1 OF 8 · INTRODUCTION

DEEP LEARNING LAB PRESENTATION

Automated Issue Classification Engine

Solving Real-World GitHub Triage with 1D-Convolutional Neural Networks

Student: Hassan **College:** RCOEM, Nagpur (CSE) **Framework:** PyTorch 2.14 & Streamlit **Live App:** gharchive.up.railway.app

What to Say (Child-Like Pitch):
"Good morning professors! Today I am presenting our Deep Learning project: an automated system that reads technical GitHub issue reports and sorts them into Bugs, Enhancements, Documentation, or Questions in under 2 milliseconds using a real PyTorch neural network."

Technical Foundation:
"Trained on 11,823 deduplicated GitHub issues from the GH Archive Lakehouse, optimizing 1,297,412 parameters with Adam and Dropout regularization."

SLIDE 2 OF 8 · PROBLEM STATEMENT

The Challenge: Millions of Unsorted Tickets

Why automated triage is necessary in modern open-source software engineering

The Real-World Bottleneck

- GitHub repositories receive **millions of issue tickets** every single day.
- Human maintainers waste hours reading titles to label them manually.
- Triage delay slows down critical bug fixes and patch releases.

> 👶 **The Mailroom Analogy (Explain like a 10-year-old):**
> "Imagine working in a busy post office where thousands of letters arrive every hour. If one person has to open and read every letter, the mail piles up. Our neural network acts like a smart sorting machine that scans the letter's headline and immediately drops it into the right basket!"

Target Categories (4 Classes)

Class	Description	Example Trigger Words
Bug	Code defect or crash	*segfault, null pointer, panic, fail*
Enhancement	New feature request	*add, support, implement, export*
Documentation	Guides & README typos	*typo, readme, tutorial, setup guide*
Question	Usage inquiries	*how to configure, where is, how can I*

> ⚙️ **Objective:** Multi-class text classification maximizing Macro-F1 across severe real-world class imbalances.

SLIDE 3 OF 8 · VERIFICATION

Academic Integrity: 100% Real Neural Network

Demonstrating true learned weights vs. fake heuristic rules

Fake Function vs. Our PyTorch Model

Aspect	Fake Rule-Based Function	Our Real Model (`IssueCNN`)
Decision Rule	`if "crash" in text: return bug`	Matrix multiplications across 1.29M weights
Learned Weights	0 bytes (None)	**5.19 MB** serialized (`cnn_weights.pt`)
Vocabulary	None	**20,075 tokens** (`vocab.json`)
Training Evidence	None (manual text)	8 epochs with **Adam optimizer** & logged loss

🗣️ **What to Say to the Professor:**
"Sir, we did not write 'if-else' statements. We trained a genuine PyTorch 1D-CNN with 1,297,412 parameters. The model weights are serialized on disk in `cnn_weights.pt` and loaded into CPU memory for sub-2ms inference."

Strict Split Isolation (Zero Data Leakage)

A rigorous audit found 551 duplicate issue records in the raw GH Archive stream. We eliminated all duplicates:

- **Total Unique Issues:** 14,780
- **Training Set (80%):** 11,823 samples
- **Validation Set (10%):** 1,478 samples
- **Test Set (10%):** 1,479 samples

Zero overlap: No issue ID or 50-token sequence exists in more than one partition.

Verified Dataset: 14,780 records · 20,075 vocabulary tokens

Slide 3

SLIDE 4 OF 8 · TOPIC-WISE ARCHITECTURE & CODE

Core Deep Learning Topics: Real PyTorch Implementation

How our model code (`src/ml/model.py`, `train.py`) executes each foundational deep learning concept

Topic 1: 1D Convolution

Practical 6 · Feature Extraction

```
self.conv = nn.Conv1d(64, 64, 3,
padding=1)
pooled = torch.max(conved, dim=2)[0]
```

What our code does: 64 filters slide across 3-word windows detecting trigger phrases (e.g. *"segfault"*). Global Max-Pooling grabs the loudest clue from the entire issue sentence.

> "Magnifying glasses scanning 3 words at a time."

Topic 2: Regularization

Practical 5 · Dropout Prevention

```
self.dropout = nn.Dropout(p=0.2)
fc_in = self.dropout(pooled)
```

What our code does: Randomly zeroes 20% of neuron outputs during each forward training pass. Prevents the 1.29M weights from memorizing exact training sentences, stopping validation loss divergence.

> "Blindfolding 20% of the player's eyes during practice."

Topic 3: Optimization

Practical 4 · Adam & Batching

```
opt = Adam(model.parameters(),
lr=1e-3)
loader = DataLoader(batch_size=64)
```

What our code does: Trains over 11,823 issues in mini-batches of 64. Adam dynamically scales learning rates per weight using momentum and variance moments for smooth, stable convergence.

> "Mountain bike with gears that adapts to steep slopes."

Topic 4: Classification Head

Practical 3 · Dense & Softmax

```
self.fc = nn.Linear(64, 4)
probs = torch.softmax(logits, dim=1)
```

What our code does: Linear layer projects 64 pooled features to 4 class logits. Softmax exponentiates and normalizes them into calibrated percentages summing to 1.00.

> "4 voting judges with percentages summing to 100%."

PyTorch Source: src/ml/model.py · src/ml/train.py · src/ml/inference.py

Slide 4

SLIDE 5 OF 8 · EXECUTION FLOW

How Words Become Decisions (Live Execution Trace)

Step-by-step tensor transformations inside the PyTorch forward pass

Tensor Lifecycle Inside `IssueCNN`

1. **Input Text:** *"Fatal segmentation fault in memory allocator"*
2. **Tokenizer:** Text mapped to 50 token IDs `[108, 412, 92, ...]` (padding with 0).
3. **Embedding:** `[1, 50]` → `[1, 50, 64]` (64-dim coordinate per word).
4. **Conv1D + ReLU:** `[1, 64, 50]` (64 filters sliding over 3-word windows).
5. **Global Max Pooling:** `[1, 64]` (one strongest feature per filter).
6. **Dropout (p=0.2):** `[1, 64]` (regularization during training).
7. **Linear Layer (FC):** `[1, 4]` (raw logits for 4 classes).
8. **Softmax:** Probability distribution across Bug, Feature, Docs, Question.

Live Output Verification

```
Input Shape: [1, 50]
Embedding: [1, 50, 64]
Conv1D: [1, 64, 50]
Max-Pool: [1, 64]
Logits: [2.84, -0.61, -1.24, -2.15]

Predicted Label: BUG (94.2% Confidence)
Latency: 1.8 ms on CPU
```

What to Say:
"This execution trace is captured live in our Streamlit dashboard on every single click, proving the forward pass executes in real-time."

SLIDE 6 OF 8 · EMPIRICAL RESULTS

Experimental Benchmarks: Real Measured Scores

Held-out evaluation across 1,479 test issues (stored in dl_report.json)

Overall Performance Comparison

Model	Test Accuracy	Macro-F1	Inference Time
Majority Class Baseline	44.69%	15.44%	0 ms
Multinomial Naive Bayes	74.65%	50.85%	< 0.5 ms
1D-CNN (PyTorch)	**77.15%**	**62.34%**	**1.8 ms**

🗣️ **What to Say:**
"Our PyTorch 1D-CNN beats Naive Bayes by +2.5% in accuracy and by a massive **+11.5% in Macro-F1**. Macro-F1 is the crucial metric because it weights all classes equally, proving the CNN captures rare categories far better than bag-of-words counting."

Per-Class Performance (Honest Breakdown)

Class	Test Count	Precision	Recall	F1-Score
Bug	661	79.4%	82.6%	**80.95%**
Enhancement	626	76.3%	80.4%	**78.29%**
Documentation	144	70.6%	58.3%	**63.88%**
Question	48	61.5%	16.7%	**26.23%**

💡 **Honest Limitation:** Questions are naturally rare in open source (only 48 test samples). While Naive Bayes scored 0% on Questions, the CNN correctly found 8, demonstrating superior semantic generalization.

Empirical Evidence: data/curated/dl_report.json

Slide 6

SLIDE 7 OF 8 · DEPLOYMENT

Interactive Dashboard & Cloud Hosting

Live Streamlit Command Center running inside Docker on Railway

Production Architecture

- **Frontend:** Streamlit 1.66 with modular tabs (`src/app/tabs/`) and dark navy theme.
- **Containerization:** Multi-stage `Dockerfile` (Python 3.11-slim, CPU PyTorch wheels).
- **Cloud Host:** Hosted live on Railway with dynamic `$PORT` binding and healthchecks.
- **Artifact Size:** Lightweight 5.19 MB model weights ensure zero-cost hobby cloud hosting.

> 🌐 **Live URL:** https://gharchive.up.railway.app
> Accessible from any mobile phone, laptop, or classroom projector!

Dashboard Capabilities

- ⚡ **Instant Quick Presets:** 4 one-click buttons to test real Bug, Feature, Docs, and Question issues.
- 📊 **Side-by-Side Comparison:** Horizontal bar charts comparing CNN probabilities vs. Naive Bayes in real time.
- 🔬 **Neural Diagnostic Inspector:** Explores token IDs, known vs. unknown words, and tensor shapes.
- 📖 **Embedded Lab Guide:** Full syllabus alignment guide rendered directly in the app.

SLIDE 8 OF 8 · DEFENSE GUIDE

Viva Voce Examination Defense Guide

Direct, confident answers to common evaluator and faculty questions

Q1: Why use 1D-CNN instead of an LSTM, RNN, or Transformer?

Answer: 1D-CNNs compute all sliding windows in parallel ($O(1)$ parallel time) rather than sequentially ($O(N)$ for LSTMs). For short issue text, local 3-word n-grams provide sufficient classification signal with 1.8 ms CPU latency, avoiding the heavy GPU and memory cost of Transformers.

Q2: What is the purpose of Dropout in your model?

Answer: `nn.Dropout(0.2)` randomly sets 20% of neuron outputs to zero during each training step. This prevents co-adaptations between neurons, forcing the network to learn robust, generalized features rather than memorizing training sentences.

Q3: Why is Macro-F1 more rigorous than overall Accuracy?

Answer: Our dataset has class imbalance (661 bugs vs. 48 questions). Accuracy gives equal weight to every sample, so predicting only majority classes looks deceptively good. Macro-F1 averages the F1 scores across all 4 classes equally, exposing performance on rare classes.

Q4: How does your Embedding layer handle Out-Of-Vocabulary (OOV) words?

Answer: Words with frequency < 2 or unseen during training map to index 1 (``). Index 0 is reserved for `` with `padding_idx=0`, ensuring padding tokens receive zero gradients and do not bias the weights.